Specification: lta_cation_subgroups.py

2026-07-21

Contents

1. Purpose and MS1 boundary **2**

2. Dependencies and provenance **2**

3. Input group model **2**

4. Public data structures **3**
- 4.1 `FiniteGroupTables` . . . 3
- 4.2 `Subgroup` . . . 3
- 4.3 `SubgroupOrbitPartition` . . . 3
- 4.4 `SubgroupConjugacyOrbit` . . . 4
- 4.5 Enumeration records . . . 4

5. Subgroup construction **4**
- 5.1 Validation . . . 4
- 5.2 Closure from generators . . . 5
- 5.3 Construction from a closed set . . . 5

6. Exhaustive subgroup enumeration **5**

7. Conjugation and policy-dependent reduction **6**

8. Subgroup orbits of sites or other objects **6**

9. Public functions **7**

10. Determinism **7**

11. Failure modes **7**

12. Complexity and scope **7**

13. Focused verification **8**

14. MS1 completion criterion **8**

18. MS7 performance revision **8**

Current authority routing: supported Group-I/II species, exact integer populations, formal charges, and charge neutrality are owned by `doc/specs/v7_13_group_i_ii_cation_extension.md`; the Al24Si24O96 charge-equivalent dry-volume model and empirical endpoint provenance are owned by `doc/specs/v7_14_group_i_ii_volume_reference_extension.md`. This specification imports those semantics where chemistry or volume enters its surface.

1. Purpose and MS1 boundary

`lta_cation_subgroups.py` implements the finite subgroup and orbit infrastructure required by the later maximum-symmetry population optimizers.

It consumes the integer operation tables and permutation actions produced by `lta_cation_symmetry.py`. It implements:

- a validated finite-group table adapter;
- subgroup representation by parent-operation index sets;
- subgroup closure from generators;
- exhaustive bounded subgroup-lattice enumeration;
- subgroup conjugation and policy-dependent conjugacy orbits;
- object-orbit partitions under a subgroup action;
- selection of the admissible conjugating group for occupation, completed exact, and strict exact modes;
- deterministic descending-order traversal;
- an explicit incomplete-search exception that withholds any maximum-symmetry claim.

MS1 does **not** resolve cation counts, color site orbits, choose off-center variants, calculate final stabilizers, or write structures. Those tasks begin in MS2-MS4.

2. Dependencies and provenance

Required packages:

`Python >= 3.10`

The module has no numerical or crystallographic dependency of its own. It accepts validated integer tables from MS0.

Permutation-group, subgroup, conjugacy, orbit, normalizer, and stabilizer terminology follows:

A. Seress, *Permutation Group Algorithms*, Cambridge University Press (2003).

The use of subgroup orbits to split a parent set of sites follows the crystallographic group-subgroup viewpoint described by:

E. Kroumova, J. M. Perez-Mato, and M. I. Aroyo, "WYCKSPLIT: a computer program for determination of the relations of Wyckoff positions for a group-subgroup pair," *Journal of Applied Crystallography* **31**, 646 (1998).

The explicit closure and exhaustive adjoining-element enumeration algorithms in this module are package-specific small-group algorithms. They are appropriate for the fixed-cell crystallographic groups expected here. They are not intended to replace Schreier-Sims methods or general-purpose computer algebra systems for large groups.

3. Input group model

Let

$$G = \{g_0, \ldots, g_{n-1}\}$$

be a finite group with integer multiplication table

$$M_{ij} = k \quad \Longleftrightarrow \quad g_i g_j = g_k,$$

inverse table

$$I_i = j \quad \Longleftrightarrow \quad g_i^{-1} = g_j,$$

and identity index e.

The multiplication convention is inherited from MS0: $g_i g_j$ applies g_j first and g_i second.

`FiniteGroupTables` validates:

1. a nonempty square multiplication table;
2. operation indices in $[0, n)$;
3. left and right identity;
4. declared two-sided inverses;
5. associativity.

MS0 already validates these properties for `SymmetryModel`. MS1 repeats the validation because it also supports synthetic abstract groups in focused tests.

4. Public data structures

4.1 `FiniteGroupTables`

```python
@dataclass(frozen=True)
class FiniteGroupTables:
    multiplication_table: tuple[tuple[int, ...], ...]
    inverse_table: tuple[int, ...]
    identity_index: int
    name: str = "finite-group"
```

Properties and methods:

```
order: int
multiply(first: int, second: int) -> int
inverse(element: int) -> int
from_group_like(group, name="finite-group") -> FiniteGroupTables
```

`from_group_like` accepts `SymmetryModel` or any object exposing the same three table attributes.

4.2 `Subgroup`

```python
@dataclass(frozen=True)
class Subgroup:
    operations: frozenset[int]
    generators: tuple[int, ...] = ()
```

The mathematical identity of a subgroup is its closed operation set. `generators` is a deterministic witness and is excluded from equality.

Properties:

```
order: int
sorted_operations: tuple[int, ...]
deterministic_key: tuple[int, tuple[int, ...]]
```

The deterministic key orders subgroups by decreasing order, then lexicographically by operation indices.

4.3 `SubgroupOrbitPartition`

```python
@dataclass(frozen=True)
class SubgroupOrbitPartition:
    subgroup: Subgroup
    action_name: str
    orbits: tuple[tuple[int, ...], ...]
    object_to_orbit: tuple[int, ...]
```

Each orbit and the orbit collection are sorted deterministically. `object_to_orbit[i]` gives the orbit index containing object i.

4.4 `SubgroupConjugacyOrbit`

```
@dataclass(frozen=True)
class SubgroupConjugacyOrbit:
    representative: Subgroup
    members: tuple[Subgroup, ...]
    conjugating_group: Subgroup
```

The representative is the lexicographically smallest operation set in the orbit. The normalizer order inside the conjugating group is

$$|N_C(H)| = \frac{|C|}{|\operatorname{Orb}_C(H)|}.$$

It is exposed through `normalizer_order`.

4.5 Enumeration records

```
@dataclass(frozen=True)
class SubgroupEnumerationReport:
    parent_group_order: int
    discovered_subgroup_count: int
    search_nodes: int
    max_search_nodes: int
    exhaustive: bool
    subgroup_orders_descending: tuple[int, ...]

@dataclass(frozen=True)
class SubgroupLattice:
    group: FiniteGroupTables
    subgroups: tuple[Subgroup, ...]
    report: SubgroupEnumerationReport
```

`SubgroupLattice` is an exhaustive deterministic collection, not a partial best-effort result.

5. Subgroup construction

5.1 Validation

`validate_subgroup(group, subgroup)` requires:

$$e \in H,$$

$$h^{-1} \in H \quad \forall h \in H,$$

and

$$h_1 h_2 \in H \quad \forall h_1, h_2 \in H.$$

Every operation index must belong to the parent group. Generator witnesses, when present, must lie in the subgroup.

5.2 Closure from generators

`subgroup_closure(group, generators)` constructs

$$\langle S \rangle,$$

where S is the supplied operation set.

The implementation starts from the identity and repeatedly adds inverses and left/right products until no operation is added. Requested generators are considered in ascending operation-index order. A generator already produced by earlier generators is omitted from the witness.

For a parent group of order n, the explicit closure cost is bounded by small polynomial work in n. The expected groups are crystallographic fixed-cell groups, so clarity and auditability are preferred over a large-group algorithm.

5.3 Construction from a closed set

`subgroup_from_operations(group, operations)` first validates the supplied set, then derives a deterministic irredundant generator witness by greedily adjoining operations in ascending order.

6. Exhaustive subgroup enumeration

`enumerate_subgroup_lattice(group, max_search_nodes=1_000_000)` uses a breadth-first adjoining-element search.

Start with the trivial subgroup

$$H_0 = \{e\}.$$

For every discovered subgroup H and every parent element $g \notin H$, construct

$$K = \langle H \cup \{g\} \rangle.$$

Add K if its operation set has not appeared before.

This is complete. For any subgroup $K \leq G$, choose a finite generating sequence $k_1, \ldots, k_m \in K$. The chain

$$\{e\} \leq \langle k_1 \rangle \leq \langle k_1, k_2 \rangle \leq \cdots \leq \langle k_1, \ldots, k_m \rangle = K$$

is reachable by the adjoining-element search, and every intermediate closure remains inside K.

A **search node** is one distinct subgroup state. This definition is stable and is recorded in the report.

If a newly discovered state would make

$$N_{\text{nodes}} > N_{\max},$$

`MaximumSymmetrySearchIncompleteError` is raised. No partial lattice is returned. A caller must not label any candidate maximum-symmetry after this exception.

After exhaustive discovery, subgroups are sorted by decreasing order and then lexicographically. `SubgroupLattice.iter_descen` exposes this order to MS2-MS3.

7. Conjugation and policy-dependent reduction

For $g \in G$ and $H \leq G$,

$$gHg^{-1} = \{ghg^{-1} : h \in H\}.$$

`conjugate_subgroup` computes this set with the integer multiplication and inverse tables, then validates it.

For an admissible conjugating subgroup $C \leq G$, the subgroup conjugacy orbit is

$$\mathrm{Orb}_C(H) = \{cHc^{-1} : c \in C\}.$$

`iter_subgroup_conjugacy_orbits` partitions an already exhaustive subgroup lattice under this action. Orbit traversal is lazy and deterministic; subgroup discovery itself is completed first so descending order and optimality accounting remain unambiguous.

The admissible group is selected by `select_admissible_conjugating_group`:

realization	variant policy	conjugating group
occupation	either	G_{site}
exact	symmetry-complete	G_{site}
exact	strict	$G_{\mathcal{V}}$

Full-parent conjugacy is not used for strict exact mode unless the strict library is globally invariant.

8. Subgroup orbits of sites or other objects

Let a validated total action be

$$\phi : G \to \mathrm{Sym}(X).$$

For $H \leq G$ and $x \in X$, the orbit is

$$H \cdot x = \{\phi(h)x : h \in H\}.$$

`subgroup_orbits(group, subgroup, action, action_name="objects")` validates:

1. one action row per parent operation;
2. every row is a permutation of the same object domain;
3. the identity row is the identity permutation;
4. action composition on all operation pairs in H:

$$\phi(h_1 h_2) = \phi(h_1) \circ \phi(h_2).$$

Because H is validated, applying every $h \in H$ once to an orbit representative gives the complete orbit.

For the future occupation optimizer, `action` is normally `SymmetryModel.physical_site_action`. The resulting orbit sizes define the indivisible population blocks under H.

9. Public functions

```
as_finite_group_tables(group, name="finite-group") -> FiniteGroupTables
validate_subgroup(group, subgroup) -> Subgroup
subgroup_closure(group, generators) -> Subgroup
subgroup_from_operations(group, operations) -> Subgroup
enumerate_subgroup_lattice(group, max_search_nodes=1_000_000) -> SubgroupLattice
conjugate_subgroup(group, subgroup, conjugating_element) -> Subgroup
subgroup_conjugacy_orbit(group, subgroup, conjugating_group) -> SubgroupConjugacyOrbit
iter_subgroup_conjugacy_orbits(group, subgroups, conjugating_group)
subgroup_orbits(group, subgroup, action, action_name="objects") -> SubgroupOrbitPartition
select_admissible_conjugating_group(symmetry_model, realization_mode) -> Subgroup
```

10. Determinism

The module must produce identical results for identical integer tables regardless of hash randomization:

- operation sets use `frozenset` for identity but are sorted before traversal;
- requested generators are processed in ascending order;
- subgroup enumeration scans parent operations in ascending order;
- lattice output uses decreasing subgroup order and lexicographic operation sets;
- conjugacy representatives use the lexicographically smallest member;
- object orbits and orbit collections are sorted by object index.

No random number generator is used.

11. Failure modes

exception	condition
`InvalidFiniteGroupError`	malformed table, invalid identity/inverse, or failed associativity
`InvalidSubgroupError`	missing identity/inverse, failed closure, bad generator, or conjugacy orbit outside supplied lattice
`MaximumSymmetrySearchIncompleteError`	subgroup node guard exceeded before exhaustive completion
`ValueError`	invalid mode, malformed action table, failed action composition, or invalid numerical limit

The incomplete-search exception carries:

```
search_nodes
max_search_nodes
discovered_subgroups
```

It is a hard correctness failure, not a warning.

12. Complexity and scope

Let $n = |G|$ and let s be the number of subgroups.

- table validation is $O(n^3)$ because associativity is checked explicitly;
- one explicit subgroup closure is polynomial in n and stores at most n operations;
- exhaustive enumeration considers at most sn adjoining-element candidates, with duplicate closures possible;
- a subgroup orbit partition costs $O(|H||X|)$ after action validation;
- a conjugacy orbit costs $O(|C||H|)$ table products, excluding validation overhead.

The implementation is intentionally optimized for transparency and small crystallographic groups. A future large-group backend may replace enumeration internally without changing the public contracts.

13. Focused verification

MS1 tests cover:

1. the complete C_4 subgroup lattice with orders $4, 2, 1$;
2. the D_4 lattice with 10 subgroups;
3. the eight D_4 subgroup conjugacy orbits;
4. the orbit-normalizer relation
$$|\operatorname{Orb}_C(H)|\,|N_C(H)| = |C|;$$
5. conjugation of reflection subgroups;
6. crystallographic grouped-site orbits from the MS0 synthetic C_4 fixture;
7. policy-correct selection of G_{site} and $G_{\mathcal{V}}$;
8. identity conjugation producing singleton subgroup orbits;
9. deterministic enumeration and conjugacy ordering;
10. rejection of non-subgroups and inconsistent actions;
11. search-node guard failure without a partial optimality claim;
12. all MS0 and legacy unique-batching regressions.

14. MS1 completion criterion

MS1 is complete when:

1. every accepted MS0 symmetry model can be converted to validated integer group tables;
2. all subgroups can be enumerated exhaustively within the configured guard;
3. policy-appropriate subgroup conjugacy orbits can be traversed deterministically;
4. grouped physical sites can be partitioned into validated subgroup orbits;
5. bounded failure cannot be mistaken for a maximum-symmetry certificate;
6. existing random, unique-batch, density, and serialization behavior remains unchanged.

The next stage, MS2, consumes these subgroup/site-orbit objects to solve exact integer occupation constraints and maximize the occupation stabilizer.

18. MS7 performance revision

Subgroup operation sets are represented internally by Python integer bit masks. Closure uses a queue of newly inserted elements and precomputed multiplication rows. Public `Subgroup`, `SubgroupLattice`, node-count, determinism, and incomplete-search contracts are unchanged.